

LAB GUIDE WITH SOLUTIONS



Module 4: Constraints, Database Design & Normalization

Module 4 - 8 Hours

Constraints, Database Design & Normalization

How to use this lab guide

Seven labs for a four-hour session. Solutions and expected output are inline.

Lab	Title	Time
4.1	Load the bad table and find the damage	25 min
4.2	Keys: identify every kind	25 min
4.3	Referential integrity and cascading rules	35 min
4.4	Normalize to 1NF	30 min
4.5	Normalize to 2NF and 3NF	40 min
4.6	Design the credit card subsystem	30 min
4.7	Denormalize deliberately, and prove it stays true	25 min

Start position: `icicibank` loaded, plus the script `00_Programme_Overview/sql/04_module04_flat_table.sql`, which creates the deliberately bad table you will fix.

Lab 4.1 - Load the bad table and find the damage

Objective: see the three anomalies in real data before you learn the theory that removes them.

Step 1 - Load it

```
\i '00_Programme_Overview/sql/04_module04_flat_table.sql'

SELECT COUNT(*) FROM branch_customer_flat;  -- 30
```

Thirty rows, one per customer, sixteen columns wide. This is what a spreadsheet looks like after somebody loads it straight into a database.

Step 2 - Look at it

```
SELECT cust_id, cust_name, branch_code, branch_name, acct_no_1, acct_no_2, products
FROM   branch_customer_flat
ORDER BY cust_id
LIMIT 6;
```

Three problems should be visible immediately: a second account in its own columns, a comma-separated product list, and branch details repeated on every row.

Step 3 - Find the first audit finding

```
SELECT branch_code,
       COUNT(DISTINCT branch_name) AS distinct_spellings,
       COUNT(*)                    AS customer_rows
FROM   branch_customer_flat
GROUP BY branch_code
HAVING COUNT(DISTINCT branch_name) > 1;
```

branch_code	distinct_spellings	customer_rows
BR03	4	4

```
SELECT DISTINCT branch_code, '[' || branch_name || ']' AS branch_name
FROM   branch_customer_flat
WHERE  branch_code = 'BR03';
```

branch_code	branch_name
BR03	[Koramangala]
BR03	[Koramangla]
BR03	[Kormangala]
BR03	[Koramangala]

Four spellings of one branch. The square brackets reveal the fourth, which differs only by a trailing space and is invisible in a normal result grid. **Any report grouped by `branch_name` now shows Koramangala four**

times, each with a fraction of the real figure.

Step 4 - Prove the update anomaly

The branch is renamed. Fix it the only way this table allows:

```
BEGIN;
UPDATE branch_customer_flat
SET   branch_name = 'Koramangala Main'
WHERE branch_name = 'Koramangala';
-- UPDATE 1
SELECT DISTINCT branch_name FROM branch_customer_flat WHERE branch_code = 'BR03';
ROLLBACK;
```

One row changed and three did not, because the other three spellings did not match the `WHERE` clause. **One fact, stored four times, requires four correct updates and offers four chances to fail.**

Step 5 - Prove the deletion anomaly

Customer 1027 closes their only account.

```
BEGIN;
DELETE FROM branch_customer_flat WHERE cust_id = 1027;
SELECT cust_phone FROM branch_customer_flat WHERE cust_id = 1027;
-- 0 rows
ROLLBACK;
```

The account is gone and so is the customer, their phone number, their city and their product list. There was nowhere else any of it was stored. **That is audit finding two.**

Step 6 - Prove the insert anomaly

ICICI Bank opens a ninth branch in Jaipur. Record it.

You cannot. The table's only key is `cust_id`, and there is no customer at the new branch yet. The branch has no independent existence in this design, so a fact about the bank cannot be recorded until an unrelated fact exists.

What just happened

You reproduced all three anomalies in ten minutes, on real data, before hearing a single normal form. Every rule in the rest of this module exists to prevent one of them.

Lab 4.2 - Keys: identify every kind

Objective: move from recognising keys to choosing them.

Step 1 - Find the candidate keys of `account`

```
SELECT COUNT(*)           AS rows,
       COUNT(DISTINCT account_id) AS distinct_ids,
       COUNT(DISTINCT account_number) AS distinct_numbers,
       COUNT(DISTINCT customer_id) AS distinct_customers,
       COUNT(DISTINCT balance) AS distinct_balances
FROM   account;
```

rows	distinct_ids	distinct_numbers	distinct_customers	distinct_balances
36	36	36	30	36

`account_id` and `account_number` are both unique across all 36 rows, so both are **candidate keys**. `customer_id` is not: 30 distinct values across 36 rows means six customers hold two accounts each.

`balance` happens to be unique too, but it is not a candidate key. Uniqueness in today's data is not the same as a guarantee. Two accounts could hold identical balances tomorrow, and nothing prevents it.

A candidate key is a rule, not an observation. Test it against the business meaning, not only against the current data.

Step 2 - Confirm what the schema declares

```
SELECT tc.constraint_name, tc.constraint_type, kcu.column_name
FROM   information_schema.table_constraints tc
       JOIN information_schema.key_column_usage kcu
         ON kcu.constraint_name = tc.constraint_name
WHERE  tc.table_name = 'account'
AND    tc.constraint_type IN ('PRIMARY KEY', 'UNIQUE', 'FOREIGN KEY')
ORDER BY tc.constraint_type;
```

`account_id` is the primary key, `account_number` the alternate key enforced by `uq_`, and `customer_id` and `branch_id` are foreign keys.

Step 3 - Build a composite key

The instalment schedule is naturally keyed on the loan plus the instalment number.

```
CREATE TABLE loan_schedule (
  loan_id          INTEGER      NOT NULL,
  installment_no    SMALLINT     NOT NULL,
  due_date         DATE         NOT NULL,
  amount_due       NUMERIC(15,2) NOT NULL,
  CONSTRAINT pk_loan_schedule PRIMARY KEY (loan_id, installment_no),
  CONSTRAINT fk_ls_loan FOREIGN KEY (loan_id)
```

```
REFERENCES loan(loan_id) ON DELETE CASCADE
);

INSERT INTO loan_schedule VALUES (5001, 1, DATE '2026-04-05', 25000.00);
INSERT INTO loan_schedule VALUES (5001, 2, DATE '2026-05-05', 25000.00);
INSERT INTO loan_schedule VALUES (5002, 1, DATE '2026-04-10', 18000.00);
```

Instalment 1 exists twice, once for each loan, and that is correct. Now try a genuine duplicate:

```
INSERT INTO loan_schedule VALUES (5001, 1, DATE '2026-04-05', 25000.00);
```

```
ERROR: duplicate key value violates unique constraint "pk_loan_schedule"
DETAIL: Key (loan_id, installment_no)=(5001, 1) already exists.
```

Neither column alone is unique. Together they are. That is a **composite key**.

Step 4 - Surrogate against natural

```
ALTER TABLE customer ADD COLUMN pan_number CHAR(10);
UPDATE customer SET pan_number = 'PAN' || LPAD(customer_id::text, 7, '0');
ALTER TABLE customer ALTER COLUMN pan_number SET NOT NULL;
ALTER TABLE customer ADD CONSTRAINT uq_customer_pan UNIQUE (pan_number);
```

The customer's identity is still `customer_id`, but the business rule that no two customers share a PAN is now enforced by the engine. **You get both.**

Ask yourself what would have happened had `pan_number` been the primary key, and a customer's PAN were later corrected after a typo at onboarding: every `account`, `loan`, `fixed_deposit` and `beneficiary` row referencing it would have to change, inside one transaction, with foreign keys fighting you the whole way.

Lab 4.3 - Referential integrity and cascading rules

Objective: see each referential action behave differently, and choose deliberately.

Step 1 - RESTRICT, the banking default

```
BEGIN;  
DELETE FROM customer WHERE customer_id = 1003;
```

```
ERROR: update or delete on table "customer" violates foreign key constraint  
"fk_account_customer_id" on table "account"  
DETAIL: Key (customer_id)=(1003) is still referenced from table "account".
```

```
ROLLBACK;
```

The database refused. That refusal is the control. Someone now has to think about what "closing" this customer actually means.

Step 2 - CASCADE, on a table where it is right

```
CREATE TABLE customer_preference (  
    customer_id INTEGER NOT NULL,  
    pref_key VARCHAR(30) NOT NULL,  
    pref_value VARCHAR(60) NOT NULL,  
    CONSTRAINT pk_customer_preference PRIMARY KEY (customer_id, pref_key),  
    CONSTRAINT fk_cpref_customer FOREIGN KEY (customer_id)  
        REFERENCES customer(customer_id) ON DELETE CASCADE  
);  
  
INSERT INTO customer_preference VALUES  
    (1003, 'STATEMENT_MODE', 'EMAIL'),  
    (1003, 'SMS_ALERTS', 'Y'),  
    (1004, 'STATEMENT_MODE', 'POST');  
  
SELECT COUNT(*) FROM customer_preference; -- 3
```

Now delete a customer who has no accounts blocking the delete. Use a scratch customer:

```
BEGIN;  
INSERT INTO customer (customer_id, first_name, last_name, date_of_birth, email,  
    phone, city, kyc_status, risk_rating, customer_since, home_branch_id)  
VALUES (1099, 'Scratch', 'Test', DATE '1990-01-01', 'scratch.test@example.com',  
    '9800000099', 'Pune', 'PENDING', 'LOW', DATE '2026-03-01', 7);  
  
INSERT INTO customer_preference VALUES (1099, 'SMS_ALERTS', 'N');  
SELECT COUNT(*) FROM customer_preference WHERE customer_id = 1099; -- 1  
  
DELETE FROM customer WHERE customer_id = 1099;  
SELECT COUNT(*) FROM customer_preference WHERE customer_id = 1099; -- 0  
ROLLBACK;
```

The preference row vanished with its customer, and that is correct: a notification preference for a customer who no longer exists is meaningless.

Step 3 - The same rule where it would be catastrophic

Imagine `account_txn` had been declared `ON DELETE CASCADE` on `account`, and `account` cascaded on `customer`. Deleting one customer row would then remove their accounts and every transaction those accounts ever had - silently, with no error, and with no record of what was removed.

```
-- What ICICI Bank actually declares
CONSTRAINT fk_account_txn_account_id FOREIGN KEY (account_id)
REFERENCES account(account_id)      -- NO ACTION: the default, and correct
```

The test: if the parent disappeared, would anybody ever need this child row again? For a preference, no - cascade. For a transaction, the regulator needs it for seven years - restrict.

Step 4 - SET NULL on the employee hierarchy

```
ALTER TABLE employee DROP CONSTRAINT fk_employee_manager_id;
ALTER TABLE employee ADD CONSTRAINT fk_employee_manager_id
    FOREIGN KEY (manager_id) REFERENCES employee(employee_id) ON DELETE SET NULL;

BEGIN;
SELECT employee_id, first_name, manager_id FROM employee WHERE branch_id = 1;
DELETE FROM employee WHERE employee_id = 501;      -- the Branch Manager
SELECT employee_id, first_name, manager_id FROM employee WHERE branch_id = 1;
ROLLBACK;
```

After the delete, employees 502 and 503 still exist with `manager_id` set to NULL. They are unassigned rather than deleted, which is exactly what should happen when a manager leaves. With `CASCADE` the whole branch team would have been deleted along with their manager.

Step 5 - The order in which you must delete

```
BEGIN;
-- Wrong order
DELETE FROM loan WHERE loan_id = 5001;
-- ERROR: violates foreign key constraint on table "loan_payment"

-- Right order: children first
DELETE FROM loan_payment WHERE loan_id = 5001;
DELETE FROM loan         WHERE loan_id = 5001;
ROLLBACK;
```

This ordering constraint matters enormously in Module 11. A migration that loads tables in the wrong order fails on every foreign key, and one that deletes in the wrong order cannot roll back cleanly.

Lab 4.4 - Normalize to 1NF

Objective: eliminate the repeating group and the multi-valued column.

Step 1 - Confirm the violations

```
-- Repeating group: how many customers use the second slot?
SELECT COUNT(*) AS customers_with_two_accounts
FROM   branch_customer_flat WHERE acct_no_2 IS NOT NULL;
-- 6

-- Multi-valued column
SELECT cust_id, products FROM branch_customer_flat WHERE cust_id IN (1003, 1005, 1027);
```

```
cust_id | products
-----+-----
  1003  | FD,LOAN,CARD
  1005  | FD,LOAN,CARD
  1027  | CARD
```

Six customers already need the second slot. **What happens to the seventh account?** The design has no answer that does not involve an `ALTER TABLE`.

Step 2 - Show why the list column is unusable

```
-- Customers holding a loan. Looks fine...
SELECT COUNT(*) FROM branch_customer_flat WHERE products LIKE '%LOAN%';
```

Now consider what breaks. This cannot use an index, so it scans the table. It would also match a product code of `HOMELOAN` or `LOANTOPUP`. There is no way to record when the customer took the product, or whether it is still active, because the column holds only names. And nothing constrains the values, so `FD`, `fd` and `Fixed Deposit` can all appear.

Step 3 - Split out the accounts

```
CREATE TABLE m4_customer_account AS
SELECT cust_id, acct_no_1 AS acct_no, acct_type_1 AS acct_type, balance_1 AS balance
FROM   branch_customer_flat WHERE acct_no_1 IS NOT NULL
UNION ALL
SELECT cust_id, acct_no_2, acct_type_2, balance_2
FROM   branch_customer_flat WHERE acct_no_2 IS NOT NULL;

SELECT COUNT(*) FROM m4_customer_account; -- 36
```

Thirty-six rows, matching the real `account` table. The repeating group became rows, and a customer may now hold any number of accounts.

Step 4 - Split out the products

```
CREATE TABLE m4_customer_product AS
SELECT f.cust_id,
       TRIM(s.product_code) AS product_code
FROM   branch_customer_flat f
       CROSS JOIN LATERAL UNNEST(STRING_TO_ARRAY(f.products, ',')) AS s(product_code)
WHERE  f.products <> 'NONE';

SELECT product_code, COUNT(*) AS holders
FROM   m4_customer_product
GROUP BY product_code
ORDER BY holders DESC;
```

product_code	holders
CARD	26
FD	20
LOAN	12

(On Oracle use `CONNECT BY` with `REGEXP_SUBSTR`; on SQL Server use `STRING_SPLIT`; on MySQL 8 use `JSON_TABLE`. The technique differs, the outcome is the same.)

Now the questions the flat table could not answer become one-liners. Which customers hold a card but no fixed deposit? How many products does the average customer hold? Both are ordinary SQL against `m4_customer_product`, and neither is expressible against a comma-separated string.

Step 5 - Add the key and the constraints the list could never have

```
ALTER TABLE m4_customer_product
  ADD CONSTRAINT pk_m4_cp PRIMARY KEY (cust_id, product_code);

ALTER TABLE m4_customer_product
  ADD CONSTRAINT ck_m4_cp_product CHECK (product_code IN ('FD','LOAN','CARD','RD'));

ALTER TABLE m4_customer_product
  ADD CONSTRAINT fk_m4_cp_customer FOREIGN KEY (cust_id) REFERENCES customer(customer_id);
```

Three guarantees that were impossible five minutes ago: no customer can be enrolled in the same product twice, no invalid product code can be stored, and no product can be held by a customer who does not exist.

Lab 4.5 - Normalize to 2NF and 3NF

Objective: remove the partial and transitive dependencies.

Step 1 - Find the transitive dependency

In `branch_customer_flat` the key is `cust_id`.

```
cust_id    -> cust_name, cust_phone, cust_city, branch_code    fine
branch_code -> branch_name, branch_ifsc, branch_city          3NF VIOLATION
```

`branch_name` describes the branch, not the customer. Confirm the redundancy:

```
SELECT branch_code, COUNT(*) AS times_branch_details_repeated
FROM   branch_customer_flat
GROUP BY branch_code
ORDER BY branch_code;
```

Each branch's name, IFSC code and city are stored once per customer of that branch. That repetition is the update anomaly, and the four spellings of Koramangala are what it produced.

Step 2 - Extract the branch

```
CREATE TABLE m4_branch AS
SELECT DISTINCT branch_code, branch_ifsc, branch_city
FROM   branch_customer_flat;

SELECT COUNT(*) FROM m4_branch;    -- 8
```

Eight rows, one per branch. Note the deliberate omission of `branch_name` from the `DISTINCT`: including it would produce eleven rows, because BR03 has four spellings. **The corrupt data cannot be normalized until it is cleaned**, which is a lesson worth more than the SQL.

```
-- Pick the authoritative name: the most common spelling per branch
CREATE TABLE m4_branch_name AS
SELECT branch_code, branch_name
FROM   (SELECT branch_code, branch_name,
              ROW_NUMBER() OVER (PARTITION BY branch_code
                                ORDER BY COUNT(*) DESC, branch_name) AS rn
        FROM   branch_customer_flat
        GROUP BY branch_code, branch_name) ranked
WHERE  rn = 1;

SELECT * FROM m4_branch_name ORDER BY branch_code;
```

(This uses a window function, which is Module 6. Run it and move on; you will understand the mechanics next week.)

For BR03 all four spellings occur once each, so the tie is broken alphabetically and `Koramangala` wins. **In a real cleanup you would not accept that.** You would take the authoritative name from the branch licence register and reject the rest, because "most common spelling" is a heuristic, not a source of truth.

Step 3 - Build the 3NF customer table

```
CREATE TABLE m4_customer AS
SELECT cust_id, cust_name, cust_phone, cust_city, branch_code
FROM branch_customer_flat;

ALTER TABLE m4_customer ADD CONSTRAINT pk_m4_customer PRIMARY KEY (cust_id);
ALTER TABLE m4_branch ADD CONSTRAINT pk_m4_branch PRIMARY KEY (branch_code);
ALTER TABLE m4_customer ADD CONSTRAINT fk_m4_customer_branch
FOREIGN KEY (branch_code) REFERENCES m4_branch(branch_code);
```

Step 4 - Prove the anomaly is gone

```
-- The update that failed in Lab 4.1 now works, once
BEGIN;
UPDATE m4_branch_name SET branch_name = 'Koramangala Main' WHERE branch_code = 'BR03';
-- UPDATE 1
SELECT * FROM m4_branch_name WHERE branch_code = 'BR03';
ROLLBACK;
```

One row, one update, no possibility of divergence. The name exists in exactly one place, so it cannot disagree with itself.

```
-- The insert anomaly is gone too: register the Jaipur branch with no customers
BEGIN;
INSERT INTO m4_branch (branch_code, branch_ifsc, branch_city)
VALUES ('BR09', 'ICIC0000709', 'Jaipur');
SELECT COUNT(*) FROM m4_branch; -- 9
ROLLBACK;
```

```
-- And the deletion anomaly: closing an account no longer erases the customer
BEGIN;
DELETE FROM m4_customer_account WHERE acct_no = 'ICIC0000009032';
SELECT cust_id, cust_name, cust_phone FROM m4_customer WHERE cust_id = 1027;
ROLLBACK;
```

The customer, their phone number and their branch are all still there. **All three anomalies removed by decomposition alone.**

Step 5 - Where 2NF applied

`branch_customer_flat` has a single-column key, so it could not have a partial dependency and was already in 2NF. To see a real 2NF violation, look at what the flat design would have become had the key been `(cust_id, acct_no)`:

<code>(cust_id, acct_no)</code>	<code>-> balance, acct_type</code>	whole key	OK
<code>cust_id</code>	<code>-> cust_name, cust_phone</code>	PART of the key	2NF violation
<code>acct_no</code>	<code>-> acct_type</code>	PART of the key	2NF violation

The customer's name would then be repeated on every account they hold, and changing it would mean updating several rows. Splitting `customer` from `customer_account` in Lab 4.4 removed exactly this.

Step 6 - Compare with the real schema

```
\d customer  
\d branch
```

The tables you have just derived are the tables you have been querying since Module 1. **This is where they came from.**

Clean up

```
DROP TABLE m4_customer_account, m4_customer_product, m4_customer,  
            m4_branch_name, m4_branch, loan_schedule, customer_preference;
```

Lab 4.6 - Design the credit card subsystem

Objective: go from a requirements paragraph to a physical schema, in the order the method prescribes.

The requirement

ICICI Bank is launching credit cards. A customer may hold several credit cards. Each card has a credit limit, a billing cycle day and a status. Every month a statement is generated for a card, showing the opening balance, purchases, payments, closing balance and a due date. Individual purchases and payments are recorded against the statement. A card may be linked to a rewards programme, and a programme has many cards enrolled in it.

Step 1 - Identify the entities

Nouns: customer, credit card, statement, transaction, rewards programme. `customer` already exists.

Step 2 - Identify the relationships and their cardinality

Relationship	Cardinality	Optionality
customer holds credit_card	One to many	A card must have a customer; a customer may have no cards
credit_card has card_statement	One to many	A statement must have a card; a new card has no statements yet
card_statement contains card_txn	One to many	A transaction must belong to a statement
credit_card enrolled in rewards_programme	Many to many	Optional both ways

Step 3 - Draw it

```
CUSTOMER ||---o{ CREDIT_CARD ||---o{ CARD_STATEMENT ||---o{ CARD_TXN
          |
          ||---o{ CARD_REWARD_ENROLMENT }o---|| REWARDS_PROGRAMME
```

Step 4 - Build it

```
CREATE TABLE credit_card (
  credit_card_id INTEGER NOT NULL,
  customer_id    INTEGER NOT NULL,
  card_number    CHAR(19) NOT NULL,
  credit_limit   NUMERIC(15,2) NOT NULL,
  billing_day    SMALLINT NOT NULL,
  issued_date    DATE NOT NULL,
  status         VARCHAR(10) DEFAULT 'ACTIVE' NOT NULL,
  CONSTRAINT pk_credit_card PRIMARY KEY (credit_card_id),
  CONSTRAINT uq_credit_card_number UNIQUE (card_number),
  CONSTRAINT fk_cc_customer FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id) ON DELETE RESTRICT,
  CONSTRAINT ck_cc_limit CHECK (credit_limit > 0),
  CONSTRAINT ck_cc_billing CHECK (billing_day BETWEEN 1 AND 28),
```

```

        CONSTRAINT ck_cc_status CHECK (status IN ('ACTIVE','BLOCKED','CLOSED','EXPIRED'))
    );

CREATE TABLE card_statement (
    statement_id INTEGER NOT NULL,
    credit_card_id INTEGER NOT NULL,
    statement_date DATE NOT NULL,
    due_date DATE NOT NULL,
    opening_balance NUMERIC(15,2) NOT NULL,
    purchases NUMERIC(15,2) DEFAULT 0 NOT NULL,
    payments NUMERIC(15,2) DEFAULT 0 NOT NULL,
    closing_balance NUMERIC(15,2) NOT NULL,
    CONSTRAINT pk_card_statement PRIMARY KEY (statement_id),
    CONSTRAINT uq_card_statement UNIQUE (credit_card_id, statement_date),
    CONSTRAINT fk_cs_card FOREIGN KEY (credit_card_id)
        REFERENCES credit_card(credit_card_id) ON DELETE RESTRICT,
    CONSTRAINT ck_cs_due CHECK (due_date > statement_date)
);

CREATE TABLE card_txn (
    card_txn_id BIGINT NOT NULL,
    statement_id INTEGER NOT NULL,
    txn_date TIMESTAMP NOT NULL,
    merchant VARCHAR(60) NOT NULL,
    amount NUMERIC(15,2) NOT NULL,
    txn_type VARCHAR(10) NOT NULL,
    CONSTRAINT pk_card_txn PRIMARY KEY (card_txn_id),
    CONSTRAINT fk_ct_statement FOREIGN KEY (statement_id)
        REFERENCES card_statement(statement_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ct_amount CHECK (amount > 0),
    CONSTRAINT ck_ct_type CHECK (txn_type IN ('PURCHASE','PAYMENT','FEE','INTEREST'))
);

CREATE TABLE rewards_programme (
    programme_code VARCHAR(10) NOT NULL,
    programme_name VARCHAR(60) NOT NULL,
    points_per_100 SMALLINT NOT NULL,
    CONSTRAINT pk_rewards_programme PRIMARY KEY (programme_code),
    CONSTRAINT ck_rp_points CHECK (points_per_100 >= 0)
);

-- The junction table resolving the many-to-many
CREATE TABLE card_reward_enrolment (
    credit_card_id INTEGER NOT NULL,
    programme_code VARCHAR(10) NOT NULL,
    enrolled_on DATE DEFAULT CURRENT_DATE NOT NULL,
    points_balance INTEGER DEFAULT 0 NOT NULL,
    CONSTRAINT pk_card_reward_enrolment PRIMARY KEY (credit_card_id, programme_code),
    CONSTRAINT fk_cre_card FOREIGN KEY (credit_card_id)
        REFERENCES credit_card(credit_card_id) ON DELETE CASCADE,
    CONSTRAINT fk_cre_programme FOREIGN KEY (programme_code)
        REFERENCES rewards_programme(programme_code) ON DELETE RESTRICT,
    CONSTRAINT ck_cre_points CHECK (points_balance >= 0)
);

```

Step 5 - Review your own design

Question	Answer here
Does every table have a primary key?	Yes, all surrogate except the junction table
Is every foreign key given a deliberate cascading rule?	Yes: RESTRICT on financial records, CASCADE on the enrolment
Is any column storing a list?	No
Is any column a fact about a different entity?	No - card_statement holds no customer name
Are the business rules in CHECK constraints?	Billing day, positive limits, valid statuses, due after statement
Which table is in a weak relationship?	card_txn , which cannot exist without its statement

uq_card_statement (**credit_card_id**, **statement_date**) is worth noting. It is an alternate key, and it enforces a real business rule: a card cannot have two statements for the same date. Without it, a re-run of the statement job would silently duplicate every statement.

Lab 4.7 - Denormalize deliberately

Objective: understand why `account.balance` exists, and build the control that keeps it honest.

Step 1 - The normalized truth

```
SELECT ROUND(SUM(CASE WHEN txn_type = 'CREDIT' THEN amount ELSE -amount END), 2)
        AS computed_balance
FROM   account_txn
WHERE  account_id = 9003 AND status = 'SUCCESS';
```

This is derivable, so by the rules of normalization `account.balance` is redundant and should not exist.

Step 2 - Why it exists anyway

```
\timing on
SELECT balance FROM account WHERE account_id = 9003;
SELECT SUM(CASE WHEN txn_type='CREDIT' THEN amount ELSE -amount END)
FROM   account_txn WHERE account_id = 9003;
\timing off
```

On 495 rows the difference is invisible. On an account with twenty years of history and 40 million rows in the table, the first is a single indexed row read and the second reads every transaction that account has ever had. A balance enquiry must answer in milliseconds, and the mobile app makes that call several thousand times a minute.

This is a justified denormalization: a named query, a measured problem, an unavoidable trade-off.

Step 3 - Maintain it correctly

```
BEGIN;
INSERT INTO account_txn
(txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no)
VALUES
(800001, 9003, TIMESTAMP '2026-03-15 11:20:00', 'DEBIT', 5000.00,
'ATM', 'ATM WDL DEMO', 'SUCCESS', 'REF800001');

UPDATE account SET balance = balance - 5000.00 WHERE account_id = 9003;
COMMIT;
```

Both statements are in **one transaction**. Atomicity guarantees they cannot diverge because of a crash. What atomicity does not protect against is a code path that inserts the transaction and forgets the update - which is why the next step exists.

Step 4 - Build the reconciliation

```
SELECT a.account_id,
       a.account_number,
       ROUND(a.balance, 2) AS stored_balance,
       ROUND(COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount
                           ELSE -t.amount END), 0), 2) AS computed_balance,
```

```
ROUND(a.balance - COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount
                                ELSE -t.amount END), 0), 2) AS difference
FROM   account a
      LEFT JOIN account_txn t ON t.account_id = a.account_id
                                AND t.status = 'SUCCESS'
GROUP BY a.account_id, a.account_number, a.balance
HAVING ABS(a.balance - COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount
                                ELSE -t.amount END), 0)) > 0.01
ORDER BY ABS(a.balance - COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount
                                ELSE -t.amount END), 0)) DESC;
```

Every account in the seed data reports a difference, because the seed generator set opening balances independently of the transactions. **In a real bank that output would be an incident.** Here it demonstrates exactly what the control is for: it detects divergence between a stored value and its source, whatever the cause.

Note `ABS(...) > 0.01` rather than `<> 0`. Reconciliations on money always need a tolerance, because rounding at different points in a pipeline produces differences of a paisa that are not errors.

Step 5 - Simulate the failure the control catches

```
BEGIN;
-- A code path that posts a transaction and forgets the balance update
INSERT INTO account_txn
(txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no)
VALUES
(800002, 9004, TIMESTAMP '2026-03-15 12:00:00', 'DEBIT', 9999.00,
 'UPI', 'UPI/P2M/DEMO', 'SUCCESS', 'REF800002');

-- Re-run the reconciliation. Account 9004's difference has moved by exactly 9999.00.
ROLLBACK;
```

Step 6 - Clean up

```
DELETE FROM account_txn WHERE txn_id IN (800001, 800002);
UPDATE account SET balance = balance + 5000.00 WHERE account_id = 9003;
```

End-of-module checklist

	Item
[]	I reproduced the insert, update and delete anomalies on real data
[]	I can identify super, candidate, primary, alternate, composite and foreign keys in a table
[]	I can explain why a phone number is not a primary key
[]	I have seen RESTRICT , CASCADE and SET NULL behave differently
[]	I can state the rule for choosing CASCADE over RESTRICT in a bank
[]	I decomposed a repeating group and a comma-separated column into 1NF
[]	I removed a transitive dependency and proved the update anomaly was gone
[]	I designed a five-table subsystem from a requirements paragraph
[]	I can justify account.balance and I have written its reconciliation

Common problems and fixes

Symptom	Cause	Fix
DISTINCT on the branch returns 11 rows, not 8	Corrupt data in the column being extracted	Clean before you normalize; agree an authoritative source
violates foreign key constraint when loading	Loading children before parents	Load parents first: branch, then customer, then account
violates foreign key constraint when deleting	Deleting parents before children	Delete children first, or choose CASCADE deliberately
Cascade deleted more than expected	CASCADE on a table that is not truly owned	Change to RESTRICT and handle closure explicitly
Junction table allows duplicate enrolments	No composite primary key	PRIMARY KEY (id_a, id_b)
Reconciliation reports differences of 0.01	Rounding at different points in the pipeline	Compare with a tolerance, not with <> 0